
1) break — *Stop the loop immediately*

What it does:

Exits the **nearest** loop entirely as soon as the condition hits.

When to use:

- You found what you needed and want to stop scanning further.
- Early termination in searches.

Example: Find the first even number, then stop

```
1  nums = [3, 5, 7, 10, 11, 14]
2
3  for n in nums:
4      if n % 2 == 0:
5          print(f"First even found: {n}")
6          break
7      print(f"Checked (odd): {n}")
8
9  print("Loop done.")
```

Output (explained):

```
1  Checked (odd): 3
2  Checked (odd): 5
3  Checked (odd): 7
4  First even found: 10
5  Loop done.
```

- As soon as 10 is found, break exits the loop; 11 and 14 are never checked.

✓ Tip: break only exits **one** level of loop. In nested loops, it does not jump out of outer loops.

2) continue — *Skip to the next loop iteration*

What it does:

Stops the current iteration and moves on to the next iteration of the loop.

When to use:

- Skip processing for certain values (e.g., invalid/empty records).
- Filter on-the-fly without building a new list.

Example: Print only non-empty student names

```
1 students = ["Anita", "", "Rahul", None, "Meera", " ", "Zoya"]
2
3 for name in students:
4     # Normalize and skip blanks
5     if not name or name.strip() == "":
6         continue
7     print(f"Present: {name.strip()}")
```

Output:

```
1 Present: Anita
2 Present: Rahul
3 Present: Meera
4 Present: Zoya
```

- Empty strings, None, and whitespace-only entries are **skipped**.
-

3) pass — *Do nothing (placeholder)*

What it does:

A no-op. It keeps the code syntactically valid when a statement is required but you don't want to do anything yet.

When to use:

- Stubs for functions/classes you'll fill later.
- Placeholder in if, for, while, or except blocks.

Example A: Placeholders while sketching structure

```
1 def process_order(order):
2     # TODO: implement later
3     pass # keeps function valid
4
5 class PaymentGateway:
6     pass # class stub
7
8 for i in range(3):
9     pass # loop stub
10
11 print("Skeleton compiles and runs.")
```

Example B: Ignore specific exceptions (explicitly)

```
1 data = ["42", "x9", "17"]
2
3 total = 0
4 for item in data:
5     try:
6         total += int(item)
7     except ValueError:
8         pass # intentionally ignore items that aren't integers
9
10 print("Total of valid integers:", total)
```

Output:

```
1 Total of valid integers: 59
```

✓ Tip: Prefer **explicit comments** with pass so future readers know it's intentional.

4) exit / sys.exit() — Stop the entire program

What it does:

Terminates the Python program **right away** (raises SystemExit under the hood).

- In **scripts**, use sys.exit() (recommended).
- exit() and quit() are meant for the **interactive shell**; avoid them in production code.

When to use:

- Fatal configuration error.
- Preconditions not met.
- Early stop after printing a message.

Example A: Input validation with sys.exit()

```
1 import sys
2
3 user_age = input("Enter age: ").strip()
4
5 if not user_age.isdigit():
6     print("Error: Age must be a positive integer.")
7     sys.exit(1) # Non-zero exit status indicates an error
8
9 age = int(user_age)
10 print(f"Thanks! Your age is {age}.")
```

Behavior:

- If input is invalid (e.g., abc), it prints the error and **terminates** the program immediately.
- Exit code 1 can be checked by calling scripts from other programs/CI.

Example B: Contrast break vs sys.exit() in loops

```
1  import sys
2
3  for n in range(1, 6):
4      if n == 3:
5          print("Using break: stop the loop, continue program.")
6          break
7      print("Loop iteration:", n)
8
9  print("Still running after break.\n")
10
11 for n in range(1, 6):
12     if n == 3:
13         print("Using sys.exit(): terminate the whole program
14         now.")
15         sys.exit(0)
16     print("Loop iteration:", n)
17
18 print("You will never see this line due to sys.exit().")
```

Output (partial):

```
1  Loop iteration: 1
2  Loop iteration: 2
3  Using break: stop the loop, continue program.
4  Still running after break.
5
6  Loop iteration: 1
7  Loop iteration: 2
8  Using sys.exit(): terminate the whole program now.
```

- The program ends before the last print because sys.exit() stops everything.

⚠ Notes:

- In some notebook environments (e.g., Jupyter/Colab), sys.exit() raises SystemExit which can appear as an exception. That's normal.
- Prefer sys.exit(code) over exit() in scripts for clarity and control.

Quick Comparison

- `break`: Exit the **nearest loop**.
 - `continue`: Skip to the **next iteration** of the loop.
 - `pass`: Do **nothing** (placeholder for future code or intentional no-op).
 - `sys.exit()`: Exit the **entire program** (recommended for scripts; choose an exit code).
-